

1.8 Exemplo de herança e plimorfismo....

Arquivo

Você é um Engenheiro de Software Sênior, especialista em Java com foco em Clean Code e experiência em ferramentas de análise de código como o SonarQuebe.

O seu objetivo é revisar e refatorar o código Java que enviarei a seguir para mitigar/melhorar os problemas apontados no relatório do SonarQube.

Por favor, analise o código Java fornecido e retorne:

O código Java refatorado, eliminando todos os Code Smells potenciais

Mostrar mais ▾



Analisei o código enviado e identifiquei alguns pontos que normalmente seriam sinalizados pelo **SonarQube** como oportunidades de melhoria relacionadas a **encapsulamento, clareza, extensibilidade e Clean Code**. O código original implementa corretamente o conceito de herança e polimorfismo, mas pode ser refinado para facilitar a manutenção e evolução do sistema.

1.8 Exemplo de herança e plimor...

Código Refatorado

Java

```
package cap1;

abstract class Empregado {

    private double salarioBase;

    protected Empregado(double salarioBase) {
        this.salarioBase = salarioBase;
    }

    public double calcularSalarioMensal() {
        return salarioBase;
    }

    protected double getSalarioBase() {
        return salarioBase;
    }

    public abstract void imprimirCargo();
}

class Secretaria extends Empregado {

    private int velocidadeDigitacao;

    public Secretaria(double salarioBase, int velocidadeDigitacao) {
        super(salarioBase);
        this.velocidadeDigitacao = velocidadeDigitacao;
    }

    public int getVelocidadeDigitacao() {
        return velocidadeDigitacao;
    }

    @Override
```

```
        public void imprimirCargo() {  
            System.out.println("Secretária");  
        }  
    }  
}
```

```
class Gerente extends Empregado {  
  
    private double bonus;  
  
    public Gerente(double salarioBase, double bonus) {  
        super(salarioBase);  
        this.bonus = bonus;  
    }  
  
    @Override  
    public double calcularSalarioMensal() {  
        return getSalarioBase() + bonus;  
    }  
  
    @Override  
    public void imprimirCargo() {  
        System.out.println("Gerente");  
    }  
}
```

```
public class Polimorfismo {  
  
    public static void main(String[] args) {  
  
        Empregado[] empregados = {  
            new Secretaria(3000.0, 120),  
            new Gerente(8000.0, 2500.0)  
        };  
  
        for (Empregado empregado : empregados) {
```

```

        empregado.imprimirCargo();
        System.out.println(
            "Salário mensal: R$ " + empregado.calcularSalarioMensal());
        System.out.println("-----");
    }
}
}

```

Alterações realizadas e Code Smells corrigidos

Alteração	Code Smell Resolvido	Princípio Aplicado
salario alterado de protected para private	Exposição desnecessária de atributos	Encapsulamento
Criação do método getSalarioBase()	Acesso controlado aos dados	Tell Don't Ask / Encapsulamento
Classe Empregado transformada em abstract	Classe concreta sem sentido de instanciar diretamente	Modelagem orientada a objetos
Método imprime() alterado para imprimirCargo()	Nome pouco descritivo	Clean Code - nomes significativos
Inclusão de construtores para inicialização obrigatória	Objetos podem ser criados em estado inválido	Fail Fast / Imutabilidade parcial
Uso de double ao invés de float	Precisão inadequada para valores monetários	Boas práticas Java

Alteração	Code Smell Resolvido	Princípio Aplicado
Uso de coleção (<code>Empregado[]</code>) e laço <code>for-each</code>	Código duplicado e pouca escalabilidade	DRY (Don't Repeat Yourself)
Remoção da instanciação direta de <code>Empregado</code>	Violação do conceito de abstração	Liskov Substitution Principle (LSP)

Explicação didática

1. Encapsulamento

No código original:

Java

```
protected float salario;
```

O atributo podia ser acessado diretamente pelas subclasses, aumentando o acoplamento e dificultando futuras alterações.

Agora:

Java

```
private double salarioBase;
```

O acesso ocorre apenas por métodos controlados.

2. Classe abstrata

Antes:

Java

```
Empregado empregado = new Empregado();
```

Um empregado genérico não representa uma entidade concreta do domínio.

Agora:

Java

```
abstract class Empregado
```

Apenas cargos reais podem ser instanciados.

3. Nomes mais claros

Métodos como:

Java

```
imprime();
```

não deixam claro o propósito.

Agora:

Java

```
imprimirCargo();  
calcularSalarioMensal();
```

A intenção fica explícita imediatamente.

4. Uso adequado do polimorfismo

Antes:

Java

```
empregado.imprime();  
secretaria.imprime();  
gerente.imprime();
```

Agora:

Java

```
for (Empregado empregado : empregados) {  
    empregado.imprimirCargo();  
}
```

Se amanhã surgir:

Java

```
class Diretor extends Empregado
```

nenhuma alteração será necessária no `main`.

Resultado esperado no SonarQube

Essa refatoração tende a reduzir alertas relacionados a:

- Fields should not be protected
- Use meaningful method names
- Abstract classes should not be instantiated
- Duplicated code
- Primitive fields should be encapsulated
- Classes should have a single responsibility
- Maintainability Rating

Além disso, o código fica mais aderente aos princípios **SOLID**, especialmente:

- **S** — Single Responsibility Principle
- **L** — Liskov Substitution Principle
- **O** — Open/Closed Principle

O código original já estava correto do ponto de vista didático para demonstrar polimorfismo, mas a versão refatorada está mais próxima do padrão utilizado em projetos corporativos reais.

       Fontes